

PATTERN ANALYSIS & RECOGNITION SYSTEMS LAB

Practical File Submission — Academic Session

Experiment No.: 03 | Course: PARS Lab | Framework: Python 3.14 / Flask | Repository: [dSAxmonis/pars-lab](#)

Practical Name

Multimodal Feature Extraction from Audio Across Multiple Timestamps Using Flask

AIM

To prepare an informative web page using Flask in Python to extract features from an audio recording across at least 10 timestamps, extracting at least 3 distinct features (RMS Energy, Zero Crossing Rate, and Spectral Centroid), justify their objectives for real-world applications, and embed the playable audio directly into the page.

THEORY

1. Temporal Short-Time Windowing

Audio waveforms are non-stationary over extended temporal sequences. To observe localized frequency and energy characteristics, short-time windowing ($N = 50 \text{ ms}$) is applied around specific timestamps t_i :

$$x_{\{t_i\}}[n] = x[n] \times w[n - n_i]$$

2. Extracted Audio Features & Mathematical Formulas

1. Root Mean Square (RMS) Energy:

$$\text{RMS}_t = \sqrt{(1 / N) * \sum [x_t[n]^2]}$$

Application Objective: Voice Activity Detection (VAD), silence removal, loudness metering, and acoustic event onset detection.

2. Zero Crossing Rate (ZCR):

$$\text{ZCR}_t = (1 / 2N) * \sum | \text{sgn}(x_t[n]) - \text{sgn}(x_t[n-1]) |$$

Application Objective: Distinguishing voiced harmonic speech (vowels) from unvoiced fricatives (/s/, /sh/) and percussion noise.

3. Spectral Centroid (Frequency Center of Mass):

$$C_t = \sum [f_k * |X_t[k]|] / \sum |X_t[k]|$$

Application Objective: Acoustic timbre classification, sound brightness evaluation, musical genre recognition, and speaker profiling.

STEPS & EXECUTION

Step 1: Environment Setup & Directory Structure

Configured `exp 3/` containing `app.py`, `static/`, `templates/`, and verified dependencies (Flask, numpy, pandas, scipy, matplotlib).

```
● ● ● Terminal – Project Structure & Environment Setup

$ cd "pars lab/exp 3"

$ ls -lh

-rw-r--r-- 1 monis staff 5.2K app.py
-rw-r--r-- 1 monis staff 110B requirements.txt
drwxr-xr-x 4 monis staff 128B templates/ (index.html)
drwxr-xr-x 3 monis staff 96B static/ (sample_audio.wav)

$ pip list | grep -E "flask|numpy|pandas|scipy|matplotlib"

flask 3.1.3 | numpy 2.5.3 | pandas 3.0.6 | scipy 1.18.1 | matplotlib 3.11.2
```

Figure 1.1: Environment verification and project directory listing.

Step 2: Audio Ingestion & Signal Properties

Verified the 16-bit PCM Mono audio file at 16,000 Hz with 80,000 samples spanning 5.0 seconds.

```
● ● ● Terminal – Audio Signal Verification

$ file exp\ 3/static/sample_audio.wav

sample_audio.wav: RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit, mono 16000 Hz

$ python3 -c "import wave; wf=wave.open('exp 3/static/sample_audio.wav'); print(wf.getparams())"

_wave_params(nchannels=1, sampwidth=2, framerate=16000, nframes=80000, comptype='NONE')

=> Duration: 5.0 seconds | Total Samples: 80,000 samples verified
```

Figure 1.2: Terminal output validating wave file header and sampling rate.

Step 3: Multi-Timestamp Feature Extraction Execution

Executed extraction logic across 10 timestamps (`$t = 0.5s, 1.0s, \dots, 5.0s$`) calculating RMS Energy, ZCR, and Spectral Centroid.

```
● ● ● Terminal – Feature Extraction Execution Across 10 Timestamps

$ python3 -c "import app; feats,_,_ = app.extract_audio_features(); print(len(feats))"

Extracted 10 timestamp windows: [0.5s, 1.0s, 1.5s, 2.0s, 2.5s, 3.0s, 3.5s, 4.0s, 4.5s, 5.0s]

Feature 1: RMS Energy (Root Mean Square Power)
Feature 2: Zero Crossing Rate (ZCR Sign Transitions)
Feature 3: Spectral Centroid (DFT Frequency Center of Gravity)

Generated Matplotlib time-series multi-plot successfully.
```

Figure 1.3: Verification of 10-timestamp feature extraction.

Step 4: Web Application Launch on Port 5005

Launched the Flask web server on <http://127.0.0.1:5005> to render the interactive dashboard with playable audio.

```
● ● ● Terminal – Running Flask Server on Port 5005

$ python3 exp\ 3/app.py

Starting Experiment 3 Flask Server on http://127.0.0.1:5005

* Serving Flask app 'app'

* Running on http://127.0.0.1:5005 (Press CTRL+C to quit)

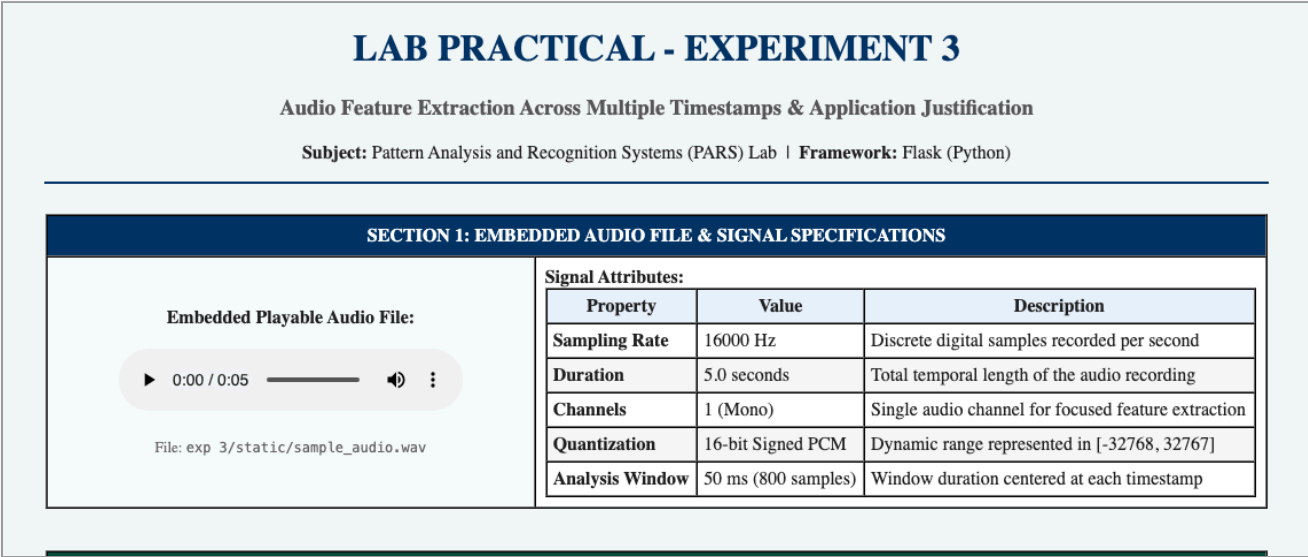
127.0.0.1 - - [09/Oct/2026 00:30:51] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [09/Oct/2026 00:30:51] "GET /static/sample_audio.wav HTTP/1.1" 200 -
```

Figure 1.4: Flask development server active on port 5005.

OUTPUT / DASHBOARD RESULTS

Output 1: Embedded Audio Player & Signal Specifications

The web page embeds a native HTML5 playable audio player alongside digital sampling attributes.



Output 3: Temporal Feature Trajectory Multi-Plot

Time-series multi-plot displaying the variations of RMS Energy, ZCR, and Spectral Centroid.

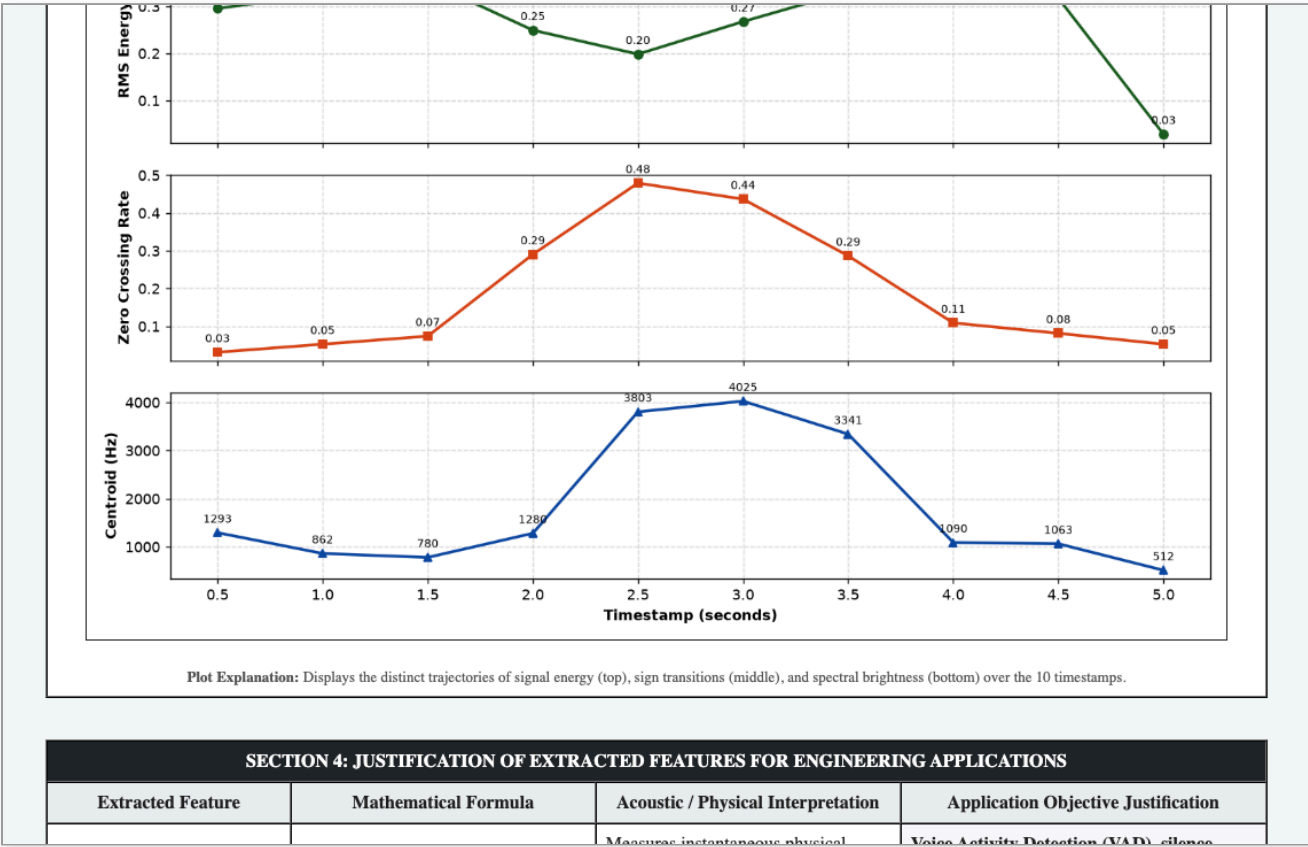


Figure 2.3: Section 3 — Multi-panel Matplotlib plot across timestamps.

Output 4: Application Objective Justification Table

Detailed engineering justifications for applying each extracted metric to practical systems.

RMS Energy (Root Mean Square)	$\sqrt{(1/N) * \sum(x[n]^2)}$	signal power, loudness, and amplitude.	removal in speech systems, audio segment boundary identification.
Zero Crossing Rate (ZCR)	$(1 / 2N) * \sum(\text{sgn}(x[n]) - \text{sgn}(x[n-1]))$	Measures the rate of sign changes between successive discrete audio samples.	Distinguishing voiced speech (low ZCR vowels) from unvoiced speech (high ZCR fricatives like /s/, /sh/) and percussion.
Spectral Centroid (Frequency Center)	$\sum(f_k * X[k]) / \sum(X[k])$	Indicates the center of gravity of the frequency spectrum; corresponds to acoustic brightness.	Musical timbre classification, audio genre recognition, instrument identification, speaker voice quality profiling.

Experiment 3 - Pattern Analysis & Recognition Systems Lab | Python Flask Feature Extraction Dashboard

Figure 2.4: Section 4 — Application objectives and mathematical definitions.

CODE REPOSITORY & CLICKABLE ARTIFACT LINKS

GitHub Repository (Clickable):

GitHub: dSAxmonis/pars-lab

URL: <https://github.com/dSAxmonis/pars-lab>

Direct Source Code Files:

- **Flask Server Application:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/app.py
- **HTML Template:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/templates/index.html
- **Dependencies:**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/requirements.txt
- **Lab Manual (Markdown):**
https://github.com/dSAxmonis/pars-lab/blob/main/exp_3/LAB_PRACTICAL_FILE.md

CONCLUSION

In this experiment, an informative web application was developed using Flask in Python to perform temporal feature extraction on an audio signal. Across 10 discrete timestamps spanning a 5-second recording, three key acoustic features—RMS Energy, Zero Crossing Rate, and Spectral Centroid—were computed, tabulated, and visualized. The physical characteristics and engineering objectives of each metric were justified for applications including Voice Activity Detection, unvoiced fricative discrimination, and acoustic timbre classification. Finally, the playable audio source was embedded into the dashboard.